Cognitum Seed — Observations & Defect Triage Discussion

Pre-ticket discussion for the development team

mondweep@dxsure.uk

2026-05-07

Cognitum Seed — Observations & Defect Triage Discussion

Status: Pre-ticket discussion — please advise which observations warrant formal defect tickets.

Purpose

During a USB-paired session on 2026-05-07, we observed a number of behaviors that may or may not be defects in (a) the seed framework, (b) the Cog Store UI, or (c) the `neural-trader` cog. Some have strong direct evidence; some may be induced by our own action sequence; some we cannot classify without source access. We would like the development team's view on which to treat as defects before opening tickets.

The triage is deliberately hedged. Where we suspect we caused the behavior ourselves, we say so. Where we cannot tell intent from external behavior alone, we frame the item as a question rather than a defect claim.

Environment

Item	Value
Hardware	Raspberry Pi Zero 2 W (per `seed.guide.overview`)
Firmware	`0.21.12`, active slot `a`; slot `b` absent; dm-verity off
MCP server	`cognitum-seed` v0.21.12, protocol `2025-11-25`, stdio↔HTTP bridge
Network	USB gadget on `en13` (`169.254.42.1/16`); no internet egress
Pairing	Paired with bearer token; roles `custody / optimizer / delivery`
Vector store	52,531 vectors at dim 8; witness chain depth 69,563
Cogs installed	`health-monitor`, `ruview-densepose` v1.2.0, `neural-trader` v1.2.0
Client	Claude Code on macOS 14.5 via `cognitum-bridge.mjs`
Device UUID	`60faaf58-bd85-418f-a203-c1b1172273a5`

Methodology

- All observations are from MCP tool calls (`seed.cogs.*`, `seed.guide.*`, `seed.framework.*`, `seed.device.*`) and a small number of direct REST calls (`GET /api/v1/apps`, `GET /api/v1/firmware/status`).
- The Cog Store UI was inspected briefly at `https://169.254.42.1:8443/store`.
- No source access; all inferences are from external behavior.
- No firmware modification; no destructive admin operations.

Triage Summary

ID	Observation	Confidence	Suggested triage
O-1	Cog Store UI Stop button does not actually terminate the cog process	High	**Likely defect**
O-2	`seed.memory.query` schema/runtime mismatch	High	**Likely defect**
O-3	Cog Store UI Start silently fails on offline-paired device	Medium	**Probable defect, cause not pinned**
O-4	UI shows registry version, not installed version	Medium	Possibly intentional UX
O-5	Running-flag desync after `config_set` → `stop` → `start` race	Low (likely induced by us)	Probably our quirk; mention only if you want to harden
O-6	`neural-trader change` blows up to ±20000% on near-zero references	Cog-side, uncertain	Cog-author question
O-7	`neural-trader` pattern matching returns `top_similarity: 0.0` always	Cog-side, uncertain	Cog-author question
O-8	`seed.cogs.*` group absent from `seed.guide.overview` taxonomy	Low	Doc-only / intentional?
O-9	`seed.cogs.console` returns bare HTTP 400 with no structured error	Low	Minor UX
O-10	`seed.sensor.list` returns `proxy_pending` (self-flagged WIP)	Known WIP	Already on roadmap presumably

---

O-1 — UI Stop button does not terminate the cog process

Suggested severity: High (user-mental-model / state integrity)

Observed behavior

1. The user clicked **Stop** on the `neural-trader` card in the Cog Store UI. The card's badge changed from "Running" to "Start ▶".

2. Some minutes later, an MCP `seed.cogs.stop` call returned `{"killed": 1, "pid": 17063, "status": "stopped"}` — the same pid that had been alive *before* the UI Stop click.

Conclusion: the UI Stop click did not kill the cog process. It appears to have only updated the framework's internal `running` flag. The kernel-level process kept running and continued to emit log entries (visible via `seed.cogs.logs`).

Reproduction (proposed)

1. Start a cog via the Cog Store UI; record its pid via `seed.cogs.list`.
2. Click Stop in the UI.
3. Immediately call `seed.cogs.list` and `seed.cogs.logs <id>`. Either: (a) `seed.cogs.list` reports `running: false` while logs continue to appear with new timestamps, or (b) calling `seed.cogs.stop <id>` afterwards returns `killed: 1` for the same pid.

Why we are confident: direct PID-level evidence; not contingent on our other actions. The MCP `stop` call would only return `killed: 1` if it found a live process to kill.

O-2 — `seed.memory.query` schema/runtime mismatch

Suggested severity: Medium

Observed behavior

Calling `seed.memory.query` with `query: "<string>"` (matching the published JSON Schema, where `query: { type: "string" }`) returns:

```
MCP error -32602: Missing 'query' array
```

Reproduction

- MCP client: send `tools/call` for `seed.memory.query` with `arguments: {"query": "neural trader market price candle", "k": 5}`.
- Result: error code `-32602`, message `Missing 'query' array`.

Interpretation: either the published schema is stale, or the runtime is over-strict. The tool description says *"Natural language or vector query string"*, implying string should be valid. We did not attempt the array form (since the schema does not document one).

O-3 — UI Start silently fails on offline-paired device

Suggested severity: Medium (user cannot start cogs from UI when offline)

Observed behavior

With the device on USB-only (no internet egress), clicking **Start** in the Cog Store UI on `neural-trader` produced no visible effect — no toast, no badge change, no new log entries. Calling `seed.cogs.start` over MCP for the same cog worked correctly and returned `{"status": "started", "pid": ...}`.

Working hypothesis (unverified): the UI's Start handler depends on the cloud cog registry for version reconciliation. `seed.cogs.available` returns `{"error": "could not fetch registry"}` on this device,

suggesting the same call path the UI uses fails. Starting a *locally installed* cog should not require registry reachability.

Caveats

- We did not capture the browser's network panel during the click.
- This may be tangled with O-5 (running-flag desync) — the UI may have read a stale `running: true` flag and short-circuited the start path. Listed as "probable" rather than "confirmed" for that reason.

O-4 — Cog Store UI shows registry/latest version, not installed version

Suggested severity: Low–Medium (UX)

Observed behavior

The UI displayed `Neural Trader v1.5.0, 36 KB`. MCP `seed.cogs.list` reports the installed version as `v1.2.0, size_kb: 20`.

Interpretation: the UI may be displaying the latest registry-known version even when the installed version is older. If intentional (surfacing an upgrade availability), an additional badge for the *installed* version would reduce confusion. If unintentional, the Cog card's primary version label should reflect what is actually running.

O-5 — Running-flag desync after `config_set` → `stop` → `start` race (likely induced by us)

Suggested severity: Self-induced; flagged only for completeness

Observed behavior

After a sequence of MCP calls — `seed.cogs.config_set` (which auto-restarts the cog: response includes `"restarted": true`), then explicit `stop`, then explicit `start` — `seed.cogs.list` reported `running: false` while the cog was demonstrably alive (pid emitting log entries; subsequent `seed.cogs.start` returned `already_running, pid 17063`).

A clean stop+start cycle without an interleaved `config_set` resolved the desync (new pid, `running: true`).

Honest caveat: this sequence is unusual. A typical user would not chain `config_set` with explicit stop and start. We attribute this primarily to our own ordering and list it for completeness only — useful as a "harden the state machine" signal rather than a user-visible defect.

O-6 — `neural-trader` change blows up to ±20000% on near-zero references

Suggested severity: Cog-author question; depends on intent

Observed behavior

In `neural-trader` log output, intermittent cycles produce values like:

`latest_change` (raw)	Rendered alert
`-209.30`	`LARGE_MOVE: change=-20929.59%`
`+140.88`	`LARGE_MOVE: change=+14088.10%`
`-61.59`	`LARGE_MOVE: change=-6159.25%`

Most cycles produce small values (`±0.01` to `±0.1`). Across the windows we observed, the spike rate was roughly 1 in 3 to 1 in 5 cycles.

Interpretation: consistent with a `(b - a) / a`-style change formula with no guard for small `a`. Could equally be intentional output of a synthetic stress-test data feed; we cannot tell from outside.

Question for the cog author: is the data feed intended to produce these extremes (e.g., to exercise alert thresholds), or is this a numerical bug? If the latter, a clamp or epsilon guard would tame it.

O-7 — `neural-trader` pattern matching never finds neighbors

Suggested severity: Cog-author question; potentially the cog's headline feature is non-functional

Observed behavior

Every cycle (across ~2 hours pre-restart and ~25 minutes post-restart) reports `similar_patterns_found: 0` and `top_similarity: 0.0` exactly. Lowering `threshold` from `2.5` to `0.7` had no effect on the result.

Interpretation: `0.0` (rather than "low") suggests the kNN search is returning zero candidates, not just no matches above threshold. Possible causes: the cog isn't writing its embeddings to a queryable collection; queries the wrong collection; mis-aligned dim; or store integration silently failing.

Question for the cog author:

- Is there a warm-up requirement (N cycles before patterns become matchable)?
- Is the cog's embedding store separate from the device's main 52k-vector store? If so, is that intentional?
- Is `top_similarity: 0.0` an empty-result sentinel, or a real similarity score?

O-8 — `seed.cogs.*` group absent from `seed.guide.overview` taxonomy

Suggested severity: Low — possibly intentional

Observed behavior

`seed.guide.overview` returns 21 tool groups. `seed.cogs.*` is not among them, despite the cog tools being functional via MCP. `seed.guide.search` for `"neural-trader"` returns 0 matches. `seed.guide.endpoints` for `group: "cogs"` returns 0 endpoints. `seed.guide.explain` reports `neural-trader` as `unknown_concept`.

Interpretation: if cogs are explicitly a community/3rd-party plugin layer, this is reasonable. If they are intended to be a first-party feature, the absence from `guide` makes them undiscoverable for new users.

O-9 — `seed.cogs.console` returns bare HTTP 400 with no structured error

Suggested severity: Low (UX)

Observed behavior

For cogs with no `allowed_commands` declared (e.g. `neural-trader`), any console call returns:

```
MCP error -32602: HTTP 400 Bad Request
```

with no body and no machine-parseable error reason.

Suggested improvement: a structured error such as

```
{"error":"console_disabled","reason":"cog declares no allowed_commands"}
```

would help client UIs render useful guidance.

O-10 — `seed.sensor.list` returns `proxy_pending`

Suggested severity: Known WIP

Observed behavior


```
{
  "note": "Tool 'seed.sensor.list' -- subsystem 'sensor' available, handler bridge in
        progress",
  "status": "proxy_pending",
  "subsystem": "sensor",
  "tool": "seed.sensor.list"
}
```


The team likely already knows. Flagging only because `seed.sensor.list` appears in the `quickStart` array of `seed.guide.overview`, which surprised us as a user.

Things we explicitly could not determine

- Where the `neural-trader` cog gets its data (no source access; no documented input endpoint; SSH not used in this session).
- Whether the cog's embedding store is the same as the device's main vector store (the `seed.memory.query` schema mismatch in O-2 blocked one investigation path).
- Whether the cog framework's `running` flag is independent of OS-level pid liveness. If it is, that cleanly explains O-1: UI Stop only updates the flag and never sends `SIGTERM`.

Questions for the development team

1. **O-1 Stop** — Is the UI Stop intended to kill the process, or only mark it stopped? If the former, we would appreciate maintainer confirmation of the repro and a ticket.
2. **O-2 Schema** — Should `seed.memory.query` accept a string query as documented, or is the runtime's array requirement correct and the schema needs updating?

3. **O-3 Start offline** — Should locally installed cogs require registry reachability to start? If not, the UI's Start path likely needs decoupling from registry reconciliation.
4. **O-4 Version display** — What version is the UI cog card meant to show — installed, latest available, or both?
5. **O-6 / O-7 (`neural-trader`)** — Are the wild change values and always-zero similarity expected outputs of the demo data feed, or numerical / store-integration bugs in the cog?
6. **O-8 Discoverability** — Are cogs first-party? If yes, would you accept a small docs change to add `seed.cogs.*` to `seed.guide.overview`?

Appendix A — Sample `neural-trader` log entries

Representative cycles, abbreviated:


```
{"candle_count":2,"latest_embedding":
[-0.0028,0.5922,-0.0028,0.5922,0.4205,-209.296,0.5950,1.0],
 "similar_patterns_found":0,"top_similarity":0.0,"signal":"neutral","signal_strength":0.0,
 "latest_change":-209.30,"latest_spread":0.5950,
 "anomalies":["HIGH_VOLATILITY: spread=0.5950","LARGE_MOVE: change=-20929.59%"],
 "timestamp":1778113944}

{"candle_count":2,"latest_embedding":
[0.5999,0.5999,0.5691,0.5691,0.7687,-0.0513,0.0308,1.0],
 "similar_patterns_found":0,"top_similarity":0.0,"signal":"neutral","signal_strength":0.0,
 "latest_change":-0.0513,"latest_spread":0.0308,
 "anomalies":["LARGE_MOVE: change=-5.13%"],
 "timestamp":1778113957}

{"candle_count":2,"latest_embedding":
[0.0041,0.5808,0.0041,0.5808,0.2934,140.881,0.5767,1.0],
 "similar_patterns_found":0,"top_similarity":0.0,"signal":"neutral","signal_strength":0.0,
 "latest_change":140.881,"latest_spread":0.5767,
 "anomalies":["HIGH_VOLATILITY: spread=0.5767","LARGE_MOVE: change=14088.10%"],
 "timestamp":1778113638}
```


The embedding shape appears to be `[v0, v1, v2, v3, v4, latest_change, latest_spread, 1.0]`, where `v0..v3` look like 2 candles' OHLC-derived features, `v4` a normalized momentum, and `v7=1.0` a constant bias slot.

Appendix B — MCP tool calls used in this session

Working: `seed.cogs.list`, `seed.cogs.logs`, `seed.cogs.config_get`, `seed.cogs.config_set`, `seed.cogs.start`, `seed.cogs.stop`, `seed.guide.overview`, `seed.guide.groups`, `seed.guide.search`, `seed.guide.explain`, `seed.guide.tutorials`, `seed.guide.endpoints`, `seed.device.status`, `seed.framework.firmware_status`, `seed.witness.chain`.

Errored / surfaced an issue: `seed.cogs.console` (HTTP 400, see O-9), `seed.cogs.available` (`could not fetch registry`, related to O-3), `seed.memory.query` (`Missing 'query' array`, see O-2), `seed.sensor.list` (`proxy_pending`, see O-10).

Appendix C — Document metadata

- Generated: 2026-05-07
- Author: session run by Mon (mondweep@dxsure.uk) with a paired Cognitum Seed (UUID `60faaf58-bd85-418f-a203-c1b1172273a5`).
- Bearer token / pairing secrets are not included in this document.